TP2 : Entrées/Sorties - Machine à états (2 séances de 3h)

L'objectif de ce TP est d'utiliser les outils et les fonctionnalités de debogage d'un environnement de programmation sur un programme simple de gestion des entrées/sorties. Nous étudierons ensuite la gestion de l'appui d'un bouton-poussoir avec la mise en évidence des rebonds pour une mise en application dans une machine à états simple avec un afficheur OLED.

- Durée : 6 heures (2 séances de TP)
- Matériels :
  - carte de test Info2 et carte SAMD21etu
  - debugger DAPLINK
- Chaîne de développement - IDE Segger Embedded Studio

2.1 Présentation de la carte Info2

La carte Info2 est une carte de test et de prototypage pour les TP Informatique Embarquée en BUT GEII : les différents composants seront utilisés durant les TPs d'Info2 du semestre 2 puis en TP Info3 au semestre 3.

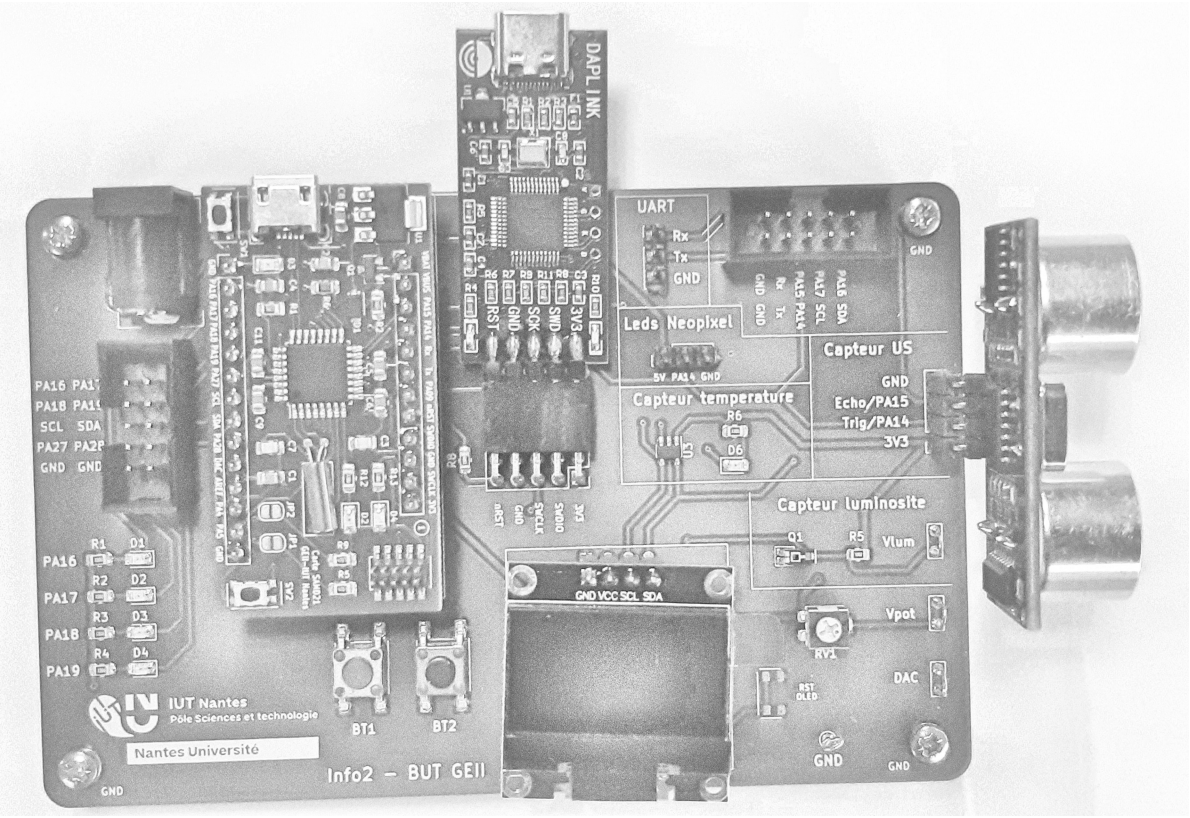


Figure 2.1 – Photo de la carte Info2

2.2 Utilisation de Segger Embedded Studio (SES)

Ouverture du projet

⚠ **IMPORTANT :** afin de gagner en rapidité, un projet Segger est fourni dans Madoc : télécharger le projet zippé « `Info2_SAMD21etu` » puis lancez l'ouverture de l'IDE **Segger Embedded Studio** en cliquant 2 fois sur `ProjetSAMD21etu.emProject`.

2.2.1 Configuration du projet

Ce projet intègre déjà la configuration de la plupart des options nécessaires :

- la sélection du circuit cible `ATSAMD21E18A` (version du SAMD21 avec 32 broches) est déjà paramétrée.
- l'outil de débogage doit par contre être spécifié :
  - ▷ Connecter le debugger DAPLINK au connecteur de la carte Info2,
  - ▷ Cliquez sur ***Target -> Connect J-Link*** pour vérifier la connection avec le debugger DAPLINK.

2.2.2 Exécution du programme fourni

```
#include "sam.h"
#include "pinAccess.h"

void attente(void) {
  volatile int i;
  for(i=0;i<800000;i++);
}

void setup() {
  for(int numLed=16;numLed<20;numLed++)
  {
    PORT_REGS->GROUP[PORTA].PORT_DIRSET = (1 << numLed);
    PORT_REGS->GROUP[PORTA].PORT_OUTSET = (1 << numLed);
  }
}

int main(void) {
  SystemInit();
  setup();

  while(1)
  {
    for(int numLed=16;numLed<20;numLed++)
    {
      PORT_REGS->GROUP[PORTA].PORT_OUT =
          (PORT_REGS->GROUP[PORTA].PORT_OUT | 0xF0000) & ~(1 << numLed);
      attente();
    }
  }
}
```

✍ Q2.2.a Ouvrez le fichier source `mainTP2.c` du projet `Info2_TP2` et exécutez-le en mode pas à pas. On précise que les 4 LEDs de la carte d'extension sont connectées en logique négative. Expliquez les opérations de masquage avec les opérateurs logiques OU, ET et NON.

2.3 Utilisation de l'afficheur OLED de la carte

S'il est indispensable de savoir utiliser les fonctionnalités de débogage d'un environnement de développement, il existe également d'autres outils pour vérifier l'exécution d'un programme informatique.
On peut utiliser par exemple un afficheur (OLED, LCD) pour afficher la valeur d'une variable, un texte pour situer l'exécution du programme ...

Vous allez utiliser dans cette partie l'afficheur OLED de la carte `Info2` pour afficher un message.
Pour ce faire, vous disposez d'une bibliothèque de fonctions intégrées dans votre projet pour manipuler cet afficheur dont les prototypes sont donnés dans le fichier « `oled.h` » du projet :

- fonctions `OLED_Init`, `OLED_Clear` pour initilisation ;
- fonction `OLED_YX` pour sélectionner les coordonnées (Ligne,Colonne) ;
- fonctions `OLED_PutChar`, `OLED_Write_String` et `OLED_Write_Integer` pour affichage des données.

```
// utilisation des fonctions de la librairie oled
#include "oledDriver.h"   //parametrage de la liaison afficheur/SAMD21
#include "oled.h"

// exemple d'utilisation de quelques fonctions
[...]
  OLED_Init();
  OLED_Clear();
  OLED_YX(0,0);
  OLED_Write_String("TP2");
  OLED_YX(1,0);
  OLED_Write_Integer(2024);
[...]
```

✍ Q2.3.a Complétez le programme précédent de façon à ce que l'allumage des leds de type « chenillard vers la gauche » soit immédiatement suivi d'un « chenillard vers la droite » et ce à l'infini.

✍ Q2.3.b On souhaite maintenant que le sens courant du chenillard soit affiché sur la première ligne de l'afficheur OLED présent sur la carte d'extension :

- ▷ Affichage de `DROITE` pour un chenillard vers la droite,
- ▷ Affichage de `GAUCHE` pour un chenillard vers la gauche.

2.4 Appui sur un bouton poussoir

On veut modifier une valeur numérique par un simple appui sur un bouton-poussoir selon l'architecture de la figure 2.2 :

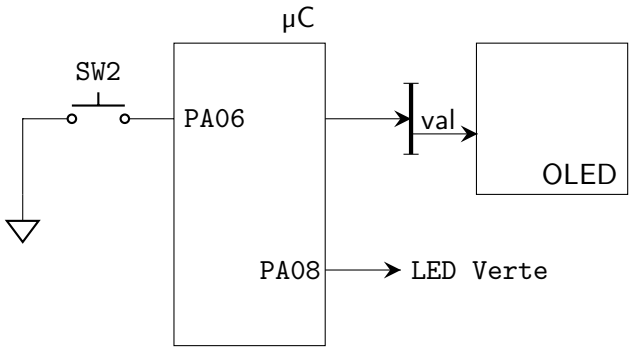


FIGURE 2.2 – Compteur de nombre d'appuis sur un poussoir

2.4.1 Structure d'une entrée

Comme le bouton-poussoir est connecté à la masse (0V), il est nécessaire de configurer le port `PA06` en entrée et utiliser une résistance de tirage interne vers le haut ou « résistance de pull-up » pour obtenir les 2 niveaux de tension.

Le schéma d'entrée d'un port est donné ci-dessous :

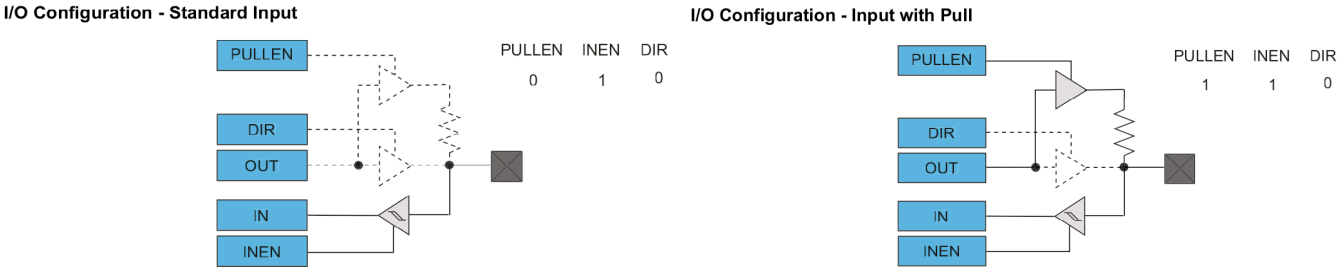


FIGURE 2.3 – Structure d'entrée d'un port

✍ Q2.4.a A partir de l'exercice effectué en TD, donner les niveaux logiques des bits `PULLEN`, `INEN`, `DIR` et `OUT` pour obtenir une entrée avec résistance de pull-up ?

✍ Q2.4.b Quel est le niveau logique de `PA06` (lecture du registre `IN`) lors d'un appui sur le bouton-poussoir ?

2.4.2 Fonction de configuration

La configuration des ports et les initialisations suivantes sont à effectuer dans la fonction `setup()` :

- Configurer `PA06` en entrée avec une résistance de pull-up et `PA08` en sortie pour indiquer un appui sur la `LED Verte`.
- Initialiser l'afficheur et afficher le texte « `TP2` » sur la première ligne.

✍ Q2.4.c Afin de simplifier l'écriture du code, on utilisera les fonctions de configuration des broches définies et documentées dans les fichiers « `pinAccess.h` » et « `carteInfo2.h` ».
En utilisant la fonction `pinMode()`, compléter la procédure `void setup()` dans le fichier `mainTP2.c` afin d'obtenir la configuration ci-dessus.

2.4.3 Fonction de détection d'un front

A chaque appui sur le bouton-poussoir, on obtient un front descendant (passage d'un niveau haut à un niveau bas). La détection d'un appui sur SW se fait par lecture de l'entrée `PA06` puis par comparaison de la valeur lue avec la valeur précédente stockée dans la variable `etatPrecedent` :

```
// Fonction de détection d'appui sur un bouton poussoir
// retourne la valeur 1 lors d'un appui sinon 0.

int DetectionFrontSW(void)
{
  int front = 0;
  static int etatPrecedent=1;

  [... A COMPLETER ...]    // comparer la valeur lue sur PA2 avec valeur etatPrecedent

  return front;
}
```

✍ Q2.4.d En vous aidant de l'exercice affectué en TD, écrire la fonction `DetectionFrontSW()` dans le fichier `mainTP2.c`.

✍ Q2.4.e Compléter la fonction `main()` pour incrémenter une valeur numérique à chaque appui sur le bouton-poussoir et afficher sa valeur sur l'écran OLED. Effectuer plusieurs tests, quelle erreur observez-vous ?

Pour filtrer les rebonds, on insère un léger délai directement dans la boucle principale du `main()` à l'aide d'une boucle `for` :

```
const int nbLoops = XX;
for (volatile int i = 0 ;  i<nbLoops ; i ++);
```

✍ Q2.4.f Insérer ce délai et déterminez expérimentalement une valeur minimale de `nbLoops` à partir de laquelle les rebonds disparaissent.

2.5 Gestion du menu d'un automate

L'application consiste à piloter les 4 leds (`PA16` à `PA19`) à partir des 3 boutons-poussoirs et utiliser l'afficheur OLED. L'organisation matérielle est représentée sur la figure 2.4.

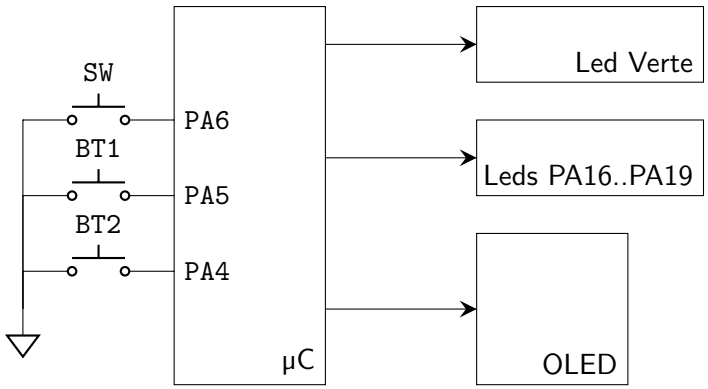


FIGURE 2.4 – Organisation matérielle de l'application de menu

L'application sera composée de deux machines à états finis (**Finite State Machine** ou FSM) :

1. une `FSM_Menu(int frontSW,int frontBT1, int frontBT2)` pour la gestion du menu afin de passer d'un menu général à des sous-menus,
2. une `FSM_OLED(int etat, int valeur)` pour l'affichage OLED.

Un premier fonctionnement peut être décrit par le graphe d'états suivant :

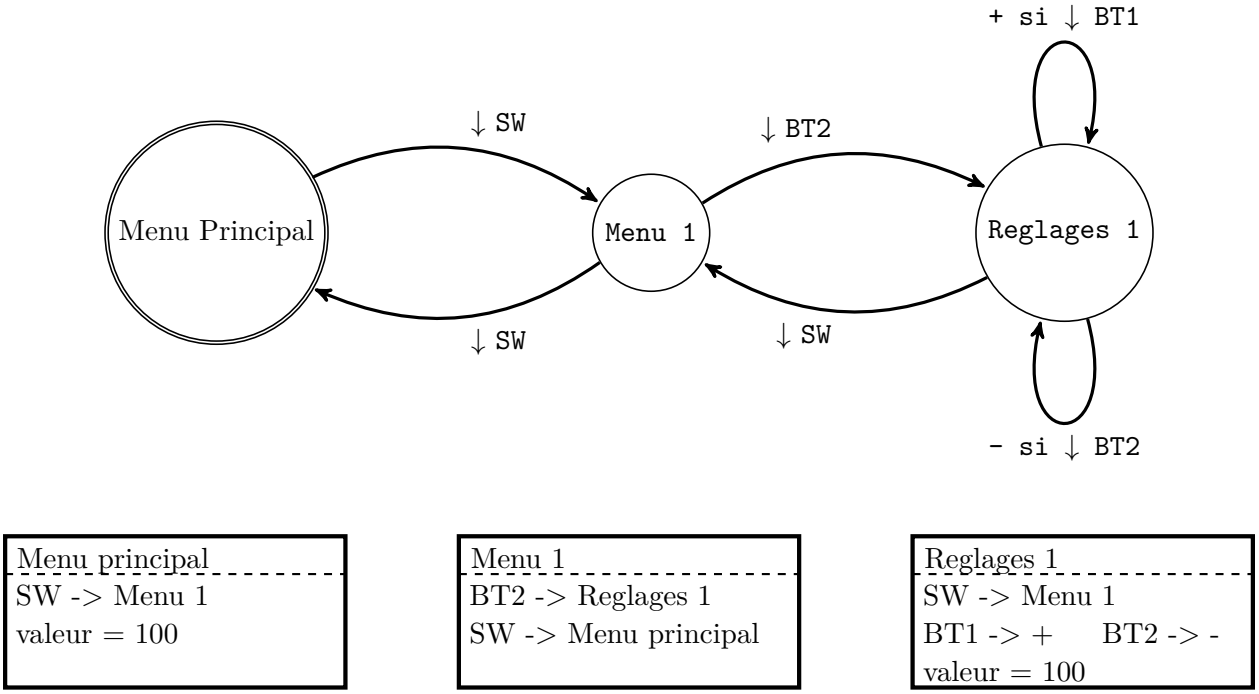


FIGURE 2.5 – Affichage OLED selon état FSM

La FSM pour la gestion du menu utilisera les états des 3 boutons-poussoirs :

- le bouton SW permet de passer du menu principal au sous-menu « `Menu 1` »,
- dans le « `Menu 1` », le bouton BT2 permet d'accéder aux « `Reglages 1` » : ce réglage consiste à incrémenter une variable « `valeur` » dont la valeur sera affichée sur l'afficheur Oled.

2.5.1 Automate version 1 avec uniquement les états `Menu pincipal` et `Menu1`

Dans la boucle principale, il doit y avoir uniquement l'appel à la FSM ainsi que le délai pour supprimer les rebonds vus dans la section précédente. Comme la machine à états finis ne doit pas être bloquante, il ne doit y avoir **AUCUNE variable globale** !

```
#define MENU_PRINC  0
#define MENU_1      1
#define REGLAGES_1  2

void setup() {
  [...]
}

int DetectionFrontSW(void) {
  [...]
}

void FSM_Menu(int valueSW) {
  static int etat = MENU_PRINC;
  static int etatPrec = MENU_PRINC;
  static int valeur=0;

  switch(etat) {
    case MENU_PRINC:
      if(valueSW) etat = MENU_1;
        OLED_YX(0,0);
        OLED_Write_String("Menu Principal");
        [...]

  [...]
}

int main() {
 SystemInit();
 setup();

 int frontSW;
 while(1)
 {
   frontSW = DetectionFrontSW();
   FSM_Menu(frontSW);
   for(volatile int i=0;i<...;i++);   //anti-rebond
 }
}
```

✍ Q2.5.a Modifier la procédure `setup()` afin d'utiliser le bouton-poussoir `SW` pour réaliser cette version 1 de l'automate.

✍ Q2.5.b Compléter la fonction `FSM_Menu()` afin de mettre en oeuvre le fonctionnement entre les 2 états « Menu Principal » et « Menu 1 ».

2.5.2 Automate version 2 avec les états `Menu1` et `Reglages1`

On souhaite maintenant :

1. détecter l'appui sur les 3 boutons-poussoirs en utilisant 3 fonctions définies dans les fichiers `carteInfo2.c` et `carteInfo2.h` :
   - `int DetectionFrontSW()`,
   - `int DetectionFrontBT1()`,
   - `int DetectionFrontBT2()`.
2. créer une deuxième machine d'états pour l'affichage sur l'OLED.

```
#include "carteInfo2.h"   //declaration des prototypes des fonctions detectionBT

void setup() {
   [...]  // paramétrages des BPi et LEDi
}

void FSM_Menu(int frontSW,int frontBT1, int frontBT2) {
  static int etat = MENU_PRINC;
  static int etatPrec = MENU_PRINC;
  static int valeur=0;

  switch(etat) {
    [...]

  if((etat != etatPrec))
    FSM_OLED(etat,valeur);
  etatPrec = etat;
}

void FSM_OLED(int etat, int valeur) {
   switch(etat) {
     [...]       // affichage variable valeur sur Oled
   }
}

int main() {
  [...]
  int frontSW, frontBT1, frontBT2 ;
  while(1)
  {
    frontSW  = DetectionFrontSW();
    frontBT1 = DetectionFrontBT1();
    frontBT2 = DetectionFrontBT2();
    FSM_Menu(frontSW,frontBT1,frontBT2);
    for(volatile int i=0;i<...;i++);     //anti-rebond
  }
}
```

✍ Q2.5.a En utilisant la même structure que la fonction `detectionFrontSW`, créer les fonctions `detectionFrontBT1` et `detectionFrontBT2` dans le fichier `carteInfo2.c`

✍ Q2.5.b Écrire la fonction `void FSM_OLED (int etat, int valeur)` permettant d'afficher sur chaque ligne de l'afficheur OLED les informations selon l'état de la FSM représentée sur la figure 2.5.

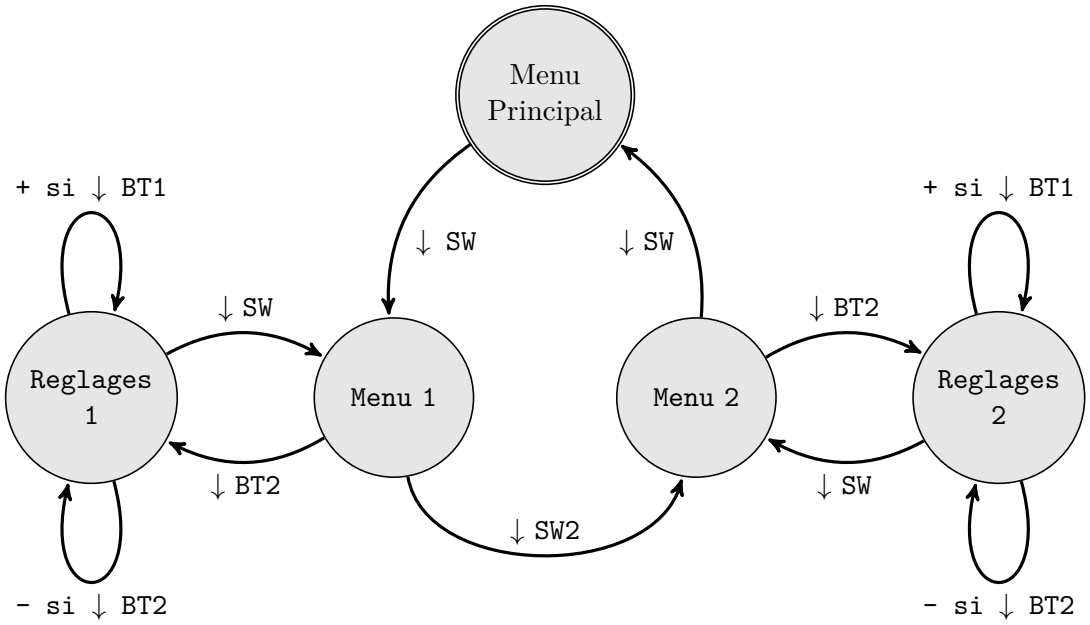
✍ Q2.5.c Compléter la fonction `void FSM_Menu(int frontSW,int frontBT1, int frontBT2)`.

2.5.3 Un menu plus compliqué

On souhaite ajouter un deuxième menu « **Menu 2** » permettant d'accéder à « **Réglages 2** » pour modifier plus rapidement la valeur de la variable `valeur` :

- ▷ dans l'état « **Réglages 2** », la variable `valeur` sera incrémentée/incrémentée de 10 entre 0 et 100,
- ▷ elle sera affichée sur la 3ème ligne de l'afficheur OLED.

Le graphe d'états modifié est le suivant :



✍ Q2.5.d Modifier l'application pour respecter ce nouveau cahier des charges.

2.5.4 BONUS : bargraphe à LEDs

Vous devez mettre en oeuvre un bargraphe à LEDs afin d'allumer les LEDs D1 à D4 en fonction de la variable `valeur` :

- 0 : aucunes LEDs allumées,
- 100 : toutes les LEDs allumées.

On utilisera le prototype suivant :

```
// Affichage sous forme de bargraphe LEDs
void bargraphe ( unsigned int value , unsigned int max ) ;
```